

Mithi AQI Predictor

An End-to-End Serverless Air Quality Forecasting System

Final Project Report

Submitted by: Ameet

Role: Intern

10Pearls — AQI Predictor Project

July-August 2026

1. Introduction

This report documents my work as an intern building an end-to-end Air Quality Index (AQI) forecasting system for Mithi, a town in the Tharparkar District of Sindh, Pakistan. The goal set out in the original project brief was to build a 100% serverless pipeline that could predict AQI for the next three days, using automated data collection, feature engineering, model training, and a live dashboard, the same structure used in real production forecasting systems, but built from scratch on free-tier tools.

Air quality forecasting matters because it gives people practical, advance warning to protect their health, especially in a region like Tharparkar, where dust storms can push pollution to hazardous levels with little visible warning. Beyond the practical value, this project was also a genuine learning exercise for me in building a real, working machine learning system end to end, not just a notebook exercise including the parts that don't show up in a typical ML course: data pipelines, automation, explainability, and debugging real infrastructure problems.

2. System Architecture

The system is built as a chain of independent, automated stages, each reading from and writing to a shared MongoDB Atlas database that acts as both the feature store and the model registry:

- Open-Meteo API: provides live and historical weather and air quality data, free and without an API key.
- Feature Pipeline: runs every hour via GitHub Actions, fetching current conditions and storing them as a new document in the features collection.
- MongoDB Atlas (features collection): the feature store, deduplicated on city and hour so re-runs never create duplicate rows.
- Training Pipeline: runs daily via GitHub Actions, pulling all historical features, training several candidate models per forecast horizon, and saving the best one.
- MongoDB Atlas (models collection): the model registry, storing each trained model as binary data directly in the database, alongside its metrics and metadata.
- Streamlit Dashboard: hosted on Streamlit Community Cloud, loads the active model and the latest data on every visit and renders the live forecast.

Storing the trained model as binary data inside MongoDB, rather than saving it to a local file, was a deliberate decision. GitHub Actions runners are torn down after every run, and the dashboard runs on a completely separate machine (Streamlit Cloud).

3. Data Pipeline

3.1 Data Source

I initially considered OpenWeather's API, but switched to Open-Meteo, since it provides both weather and air quality data: current, historical, and forecast. From a single provider, entirely

free and without requiring an API key. This simplified the whole pipeline considerably, since every component (live pipeline, backfill, and live prediction) could pull from the same source.

3.2 Feature Store Design

Every hourly reading is stored as a single MongoDB document containing pollutant levels (PM2.5, PM10, CO, NO2, SO2, O3), weather conditions (temperature, humidity, pressure, wind speed), time-based features (hour, day, month, day of week), and a derived AQI change-rate feature. Each document is upserted using a unique key of city and hour, so if the hourly pipeline ever runs twice in the same hour, it updates the existing row instead of creating a duplicate.

3.3 Historical Backfill

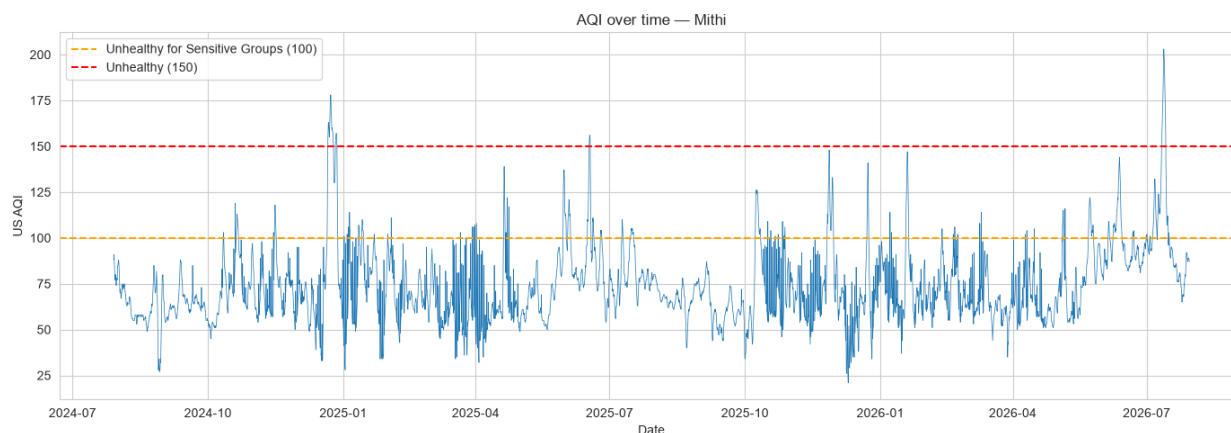
To build a training dataset, I backfilled two years of hourly historical data (roughly 17,500 rows) by calling Open-Meteo's historical weather and air quality archive endpoints in monthly chunks, to keep each API call small and avoid timeouts.

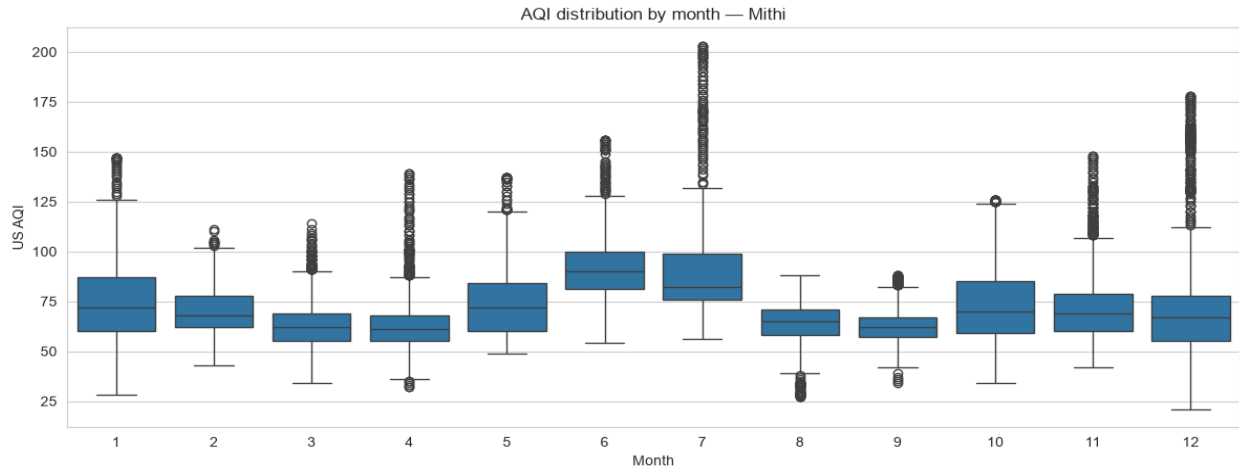
4. Exploratory Data Analysis

Before modeling, I explored the data to understand what I was actually working with, rather than assuming patterns from general knowledge. This turned up several findings specific to Mithi that directly shaped later decisions.

4.1 A Different Seasonal Pattern Than Expected

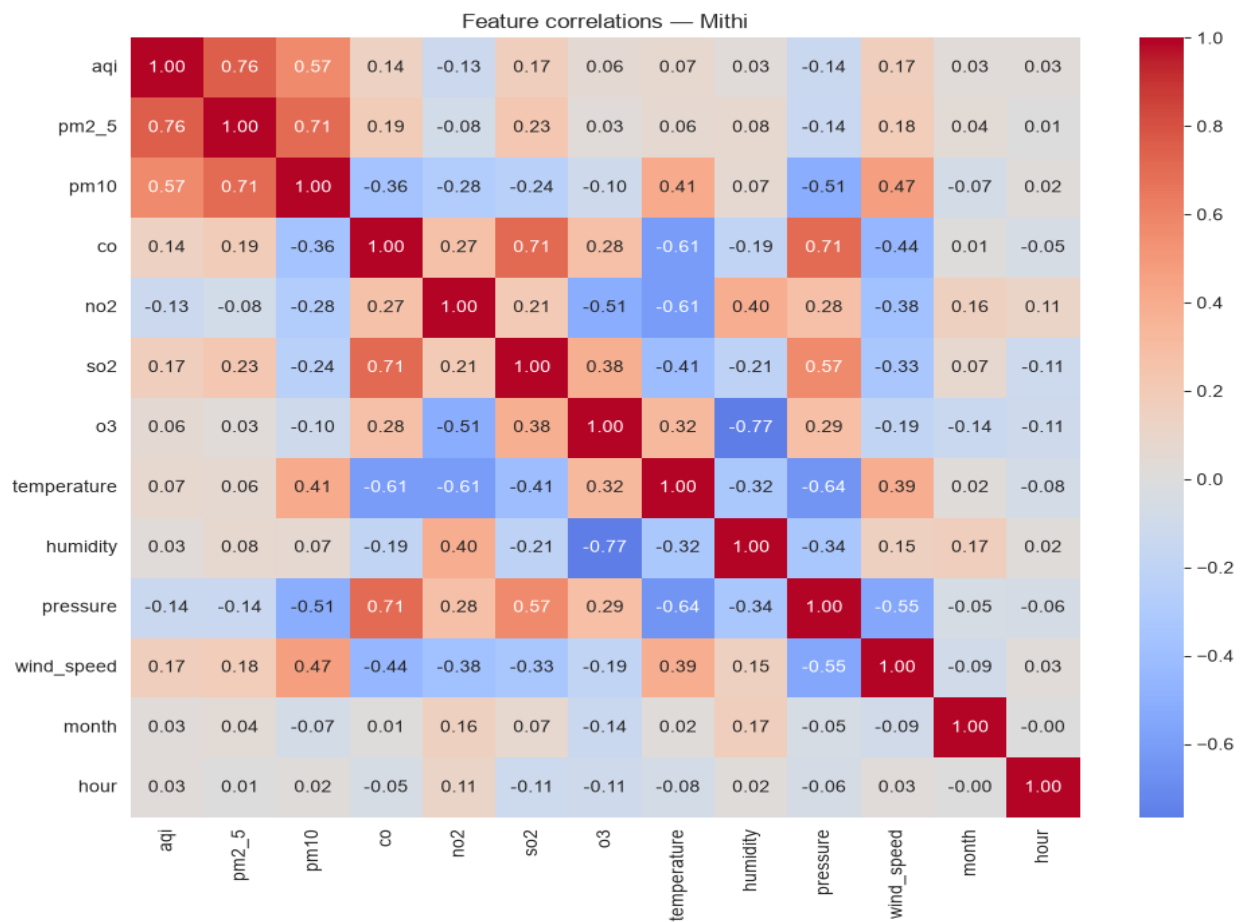
AQI in Mithi peaks in the pre-monsoon months of June and July, and drops sharply once monsoon rains arrive in August and September. This is the opposite seasonal shape from a typical urban-pollution-driven city, where winter temperature inversions usually trap pollution. Mithi sits in the Thar Desert, so its pollution is dominated by dust storms rather than traffic or industrial emissions, and dust storms are more frequent in the hot, dry months before the monsoon arrives.





4.2 Correlation Analysis

PM2.5 showed the strongest single correlation with AQI (0.76). Interestingly, wind speed correlated positively with pollution levels (0.17 to 0.47 depending on the pollutant), the opposite of what I initially expected. In an urban setting, wind usually disperses pollution and lowers AQI. Here, wind appears to kick up desert dust and raise it.



4.3 Validating Extreme Values

The dataset contained some very high AQI spikes (up to 203). Rather than treating these as errors to remove, I checked whether they were genuine events by looking at the ten highest AQI hours directly. They clustered on a single date, with PM10 far exceeding PM2.5, the signature of a dust storm, not a sensor glitch. I kept these values in the dataset, since they are exactly the kind of event the hazardous AQI alert feature is meant to catch.

5. Feature Engineering

This was the single most important change I made to this project, and it came directly from an honest look at weak initial results, which I discuss further in Section 11.

My first working version of the model predicted AQI three days ahead using only the current day's weather and pollution readings as inputs. This performed poorly, because today's weather has very little predictive power for what the weather will actually be three days from now.

The redesigned feature set uses three distinct types of inputs, each chosen for a specific reason:

- Pollutant levels from right now: the most recent, genuinely known information about air quality.
- Forecasted weather for the target day: fetched live from Open-Meteo's forecast API at prediction time. During training, since a forecast archive of past forecasts does not exist, I used the actual historical weather that occurred on the target day as a stand-in, since a 1-3 day forecast is normally close to what actually happens.
- Calendar features for the target day (hour, day, month, day of week): these are always exactly knowable in advance, regardless of horizon, so I aligned them to the target time rather than today's time.

I want to be transparent about one limitation of this approach: because training uses the actual historical weather (perfect information) as a stand-in for a forecast (which carries some genuine error), the backtested accuracy reported in this document is likely slightly higher than what the system will achieve in real-world use, where the live forecast used for prediction will not be perfectly accurate. This is a known, industry-standard limitation of this technique, not a flaw unique to this implementation, and I chose to disclose it rather than leave it unstated.

6. Modeling & Methodology

6.1 Candidate Algorithms

I trained and evaluated five candidate algorithms at each forecast horizon: Linear Regression, Ridge Regression, Random Forest, Gradient Boosting, and XGBoost.

6.2 Train/Test Split

Since this is time-series data, I used a chronological split (the earliest 85% of the data for training, the most recent 15% for testing) rather than a random split. A random split would let the

model train on data from after the test period, which does not reflect how the system will actually be used, in reality, a forecast can only ever use information from the past. I also aligned each training example strictly by timestamp arithmetic, so that any gaps in the hourly data could not silently shift feature-target pairs out of alignment.

6.3 Multi-Horizon Models

Rather than training one model to predict '3 days ahead' broadly, I trained a separate model for each of Day 1 (24 hours), Day 2 (48 hours), and Day 3 (72 hours), independently selecting the best-performing algorithm for each.

7. Results

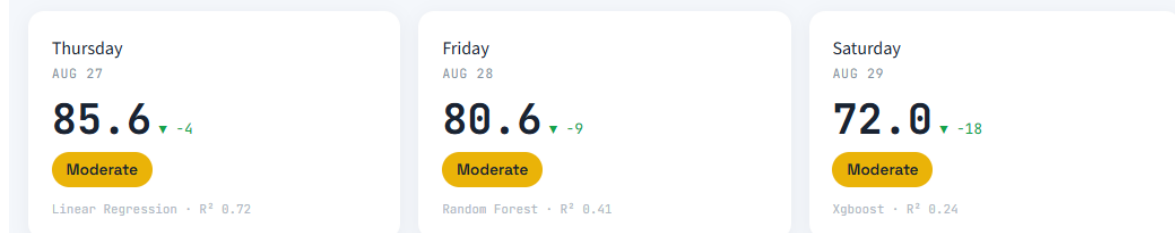
The table below shows the best-performing model at each horizon, evaluated on the held out 15% of the data:

Horizon	Best Model	RMSE	MAE	R ²
Day 1 (24h)	Linear Regression	13.04	9.13	0.718
Day 2 (48h)	Random Forest	18.94	12.28	0.407
Day 3 (72h)	XGBoost	21.43	14.51	0.240

Accuracy degrades with forecast distance, which is expected and well-documented in air quality forecasting research generally. Day 1 benefits from highly accurate near-term weather forecasts and strong AQI persistence, while Day 3 involves genuinely more uncertainty in both weather and pollution dynamics.

One finding I did not expect: the best algorithm changes by horizon. Linear Regression wins at Day 1, where the relationship between forecasted weather and AQI appears close to direct and near-term forecasts are highly reliable. Tree-based models (Random Forest, then XGBoost) win at Days 2 and 3, where relationships appear noisier and less linear. I did not anticipate this before running the comparison, which is exactly why I tested multiple algorithm rather than assuming one.

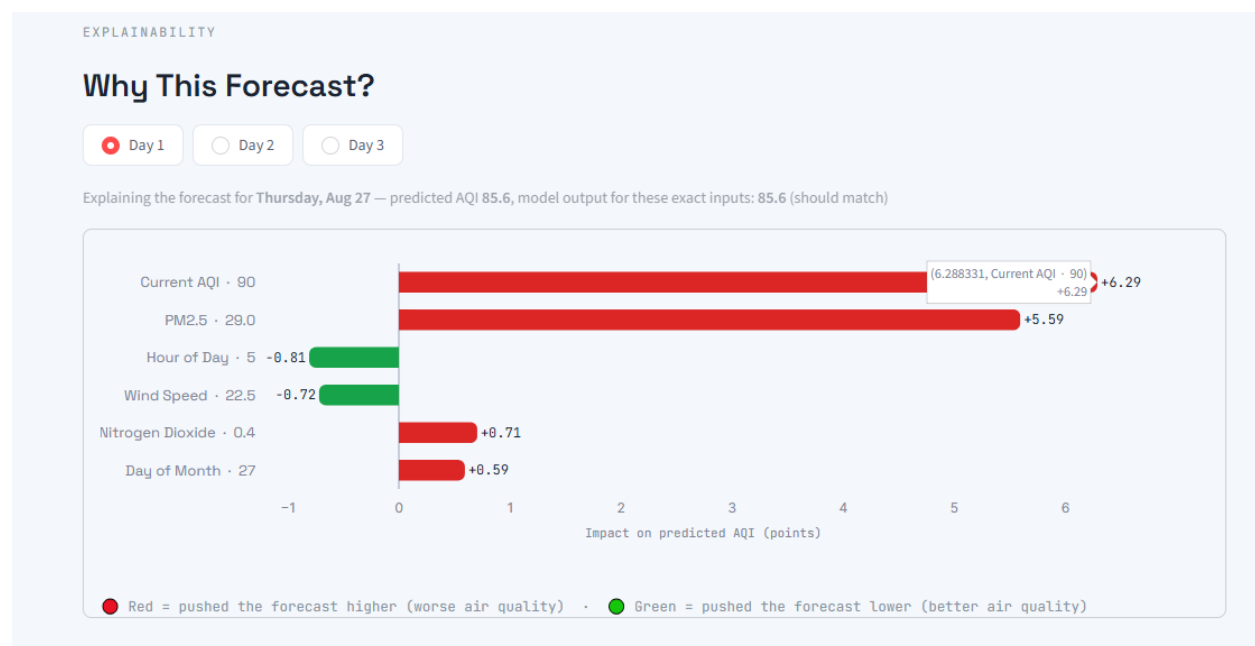
Next 3 Days



8. Explainability (SHAP)

Each forecast on the dashboard includes a SHAP breakdown, showing which specific features pushed that day's prediction up or down, and by how much. I used SHAP's unified Explainer API with a background sample of historical data, since this supports both the tree-based models (Random Forest, XGBoost) and the linear model (Linear Regression) used across the three horizons, through the same code path.

To confirm the explanation is trustworthy, I added a visible self-check directly on the dashboard: it displays both the predicted AQI and the model's output when re-run on those exact same inputs, so the two numbers can be visually compared. I verified across all three days that the target dates correctly increment (for example, Aug 14, Aug 15, Aug 16) and that the two numbers match exactly in every case.



9. Automation (CI/CD)

Two GitHub Actions workflows automate the system without requiring any manual intervention:

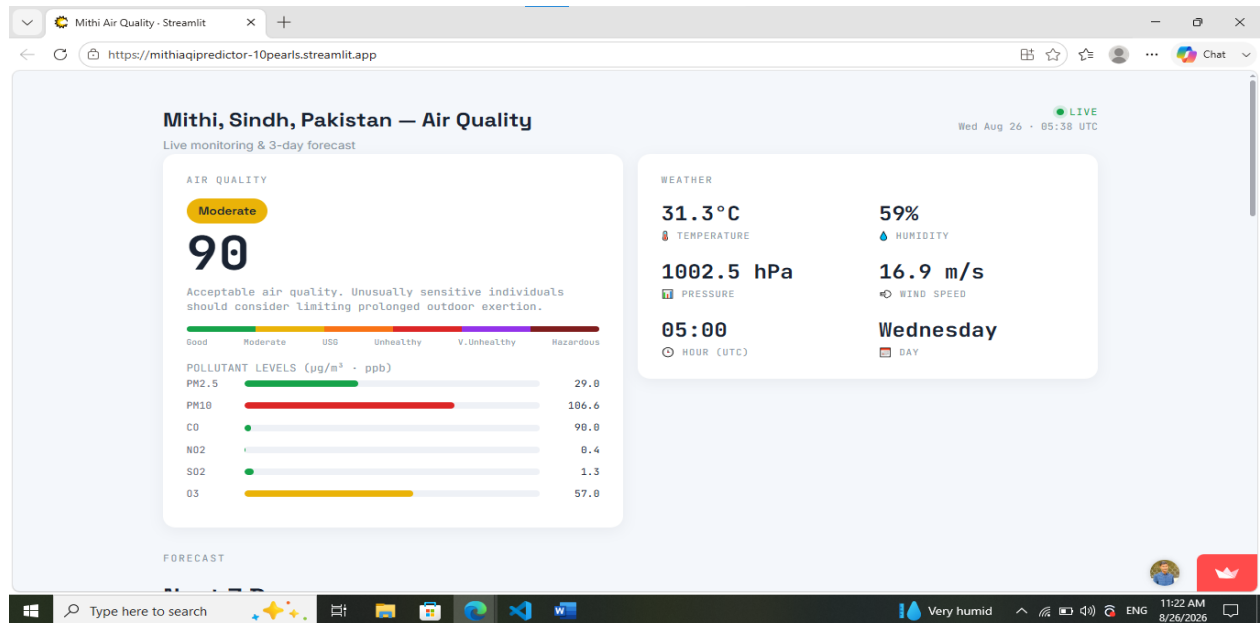
- Feature pipeline: runs every hour, fetching live conditions and writing a new row to the feature store.
- Training pipeline: runs once daily, retraining all three horizon models on the latest available data and updating the active model registry.

The training pipeline also includes a retention policy: after each run, only the three most recent model versions per horizon are kept, with older ones automatically deleted. Early in the project, before this was in place, a training run generated an oversized Random Forest model (roughly 35 MB) that exceeded MongoDB's per-document size limit and crashed the pipeline. I fixed this by compressing the serialized model and adding a size check before saving, and separately added

the retention policy after realizing that, left unattended, daily retraining would eventually grow the database beyond its free-tier storage limit.

10. Dashboard & Alerts

The Streamlit dashboard displays current air quality and weather conditions in two separate panels, a 3-day forecast with day names and dates, a pollutant breakdown, the SHAP explanation described above, a 14-day historical trend chart, and a model performance table showing RMSE, MAE, and R² per horizon.



10.1 Hazardous AQI Alerts

The dashboard shows a two-tier alert banner, checking both current conditions and all three forecasted days for the worst value across all of them:

- Caution (AQI 100 or above) — sensitive groups (children, elderly, respiratory or heart conditions) advised to limit prolonged outdoor exertion.
- Warning (AQI 150 or above) — general public advised to limit outdoor activity.

I chose a two-tier system after considering the tradeoff directly: a single threshold at 150 would leave sensitive groups uninformed during a real window where the EPA's own guidance says they should already be taking precautions.

10.2 Health Guidance

Alongside the alert, the dashboard shows specific, actionable guidance text (for example, recommending an N95 mask at higher AQI levels) rather than only a category label. This text is generated from a single shared source used by both the main air quality panel and the alert banner, so the two can never show inconsistent advice.

11. Challenges & Debugging

11.1 Feature Store: From Hopsworks to MongoDB

The original brief suggested Hopsworks as a feature store option. I attempted to set this up first, but ran into significant setup and integration difficulties that were consuming disproportionate time. I switched to MongoDB Atlas as a alternative feature store and model registry. This turned out to be the right call: MongoDB's free tier was simple to set up, and by designing a clear document schema (upserted on city and hour, with a unique index) I was able to replicate the important parts of a feature store deduplication, structured storage, and queryability without the added complexity Hopsworks introduced for this project's scale.

11.2 Weak Initial Model Accuracy

My first working model produced an R^2 of only around 0.15 to 0.48 across the three horizons. Rather than accepting this, I investigated why, identified that the feature set was not forward-looking, and rebuilt it using forecasted weather instead, which raised R^2 substantially, to 0.72, 0.41, and 0.24 at Days 1, 2, and 3 respectively.

11.3 Data Storage and Infrastructure Issues

Over the course of the project I encountered and resolved a number of real infrastructure problems, including: a MongoDB sort-memory-limit crash during automated cleanup, fixed by adding a proper index; a DNS resolution failure specific to MongoDB's connection format, fixed by switching to a public DNS resolver; an oversized model exceeding MongoDB's document size limit, fixed with compression and a pre-save size check; and an XGBoost version-compatibility warning, fixed by switching to XGBoost's native save format instead of generic pickling.

12. Bonus

In addition to the core deliverables outlined in the project brief, I implemented one extensions that went beyond what was asked for:

- Actionable health guidance text tailored to each AQI category (for example, recommending an N95 mask or limiting outdoor exertion), shown alongside the current conditions and the hazardous alert banner, rather than only a generic warning message.

13. Limitations & Future Work

- Training uses actual historical weather as a stand-in for forecasted weather; live predictions use genuine forecasts, which carry some error not present during training. Real-world accuracy may be marginally lower than the backtested numbers reported here, particularly at Day 3.

- Day 3 accuracy remains modest (R^2 0.24). This reflects the genuine difficulty of multi-day air quality forecasting broadly, not a specific flaw in this implementation, but it is an honest limit of the current system.
- Ammonia (NH3) is excluded from the feature set, since Open-Meteo's air quality model does not report it for this location.
- The system currently supports one city at a time; extending it to a dashboard city-selector would be a natural next step, since the underlying data model already supports multiple cities.
- Prediction uncertainty (confidence intervals) is not currently shown alongside point forecasts, which would add useful context for users.

14. Conclusion

This project gave me hands-on experience with the full lifecycle of a real machine learning system: data collection, feature engineering, model training and comparison, explainability, automation, and deployment as well as the debugging and iteration that goes into making a system actually reliable, not just functional once. The most important lesson for me was that an honest look at weak initial results led directly to the biggest improvement in the project, and that a piece of critical feedback from my mentor uncovered a real correctness bug that a purely visual check would have missed. I believe both of those moments were more valuable to my learning than anything that worked correctly on the first attempt.

Technology Stack

Component	Technology
Data source	Open-Meteo (Weather + Air Quality APIs)
Feature store / model registry	MongoDB Atlas (free tier)
Machine learning	scikit-learn, XGBoost
Explainability	SHAP
Automation	GitHub Actions
Dashboard	Streamlit, Plotly
Hosting	Streamlit Community Cloud